

IFN647 Week 12 Review Questions

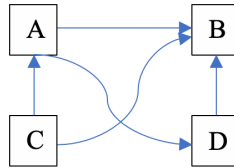
Question 1 (PageRank)

A Web graph $G = (P, L)$ consists of Web pages (vertices) and links (edges). The PageRank procedure takes a Web graph G as input and then outputs the better PageRank estimate PR using the following equation

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \cdot \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

where B_u is the set of pages that point to u , and L_v is the number of outgoing links from page v .

The following figure $G = (P, L)$ is a Web graph, where $P = \{A, B, C, D\}$ and $L = \{(A, B), (A, D), (C, A), (D, B), (C, B)\}$.



Assume the initial estimate of each Web page is equally, i.e., $PR(A) = PR(B) = PR(C) = PR(D) = 0.25$; and $\lambda = 0.15$. Calculate the PageRank estimate (PR value) of each Web page after running the first iteration of procedure PageRank(G).

Question 2 (Attention to BERT)

In Week 3, we discussed BERT-base word embeddings, where a pre-trained tokenizer and model can be loaded as follows:

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
```

Week 3 Review Question 7 demonstrated how to use a standard pre-trained BERT-base model to encode two input texts and generate their corresponding token-level (word) embeddings. The code covered both the initial encoding process and the final step of obtaining word embeddings.

In this week, you will extend this code to better understand how the `attention_mask` is used to obtain encoder outputs. Your implementation may include the following steps:

1. Tokenize an input text
2. Generate initial embeddings (i.e., encoder inputs), and optionally print the representation of the first token

3. Pass the initial embeddings to the BERT encoder using the `attention_mask` to obtain the encoder outputs
4. Print the representation of the first token again to observe the differences

Question 3 (Application of unsupervised contrastive learning)

How can pseudo-relevant feedback techniques be effectively integrated with contrastive learning to identify and construct potential positive samples in a learning framework? Building on the concepts covered in this unit, can you design a clear, step-by-step pipeline that demonstrates how pseudo-correlations are generated, refined, and incorporated into contrastive training? Additionally, can you illustrate this process with a concrete example for a given query Q , showing how candidate samples are selected, evaluated, and used to form positive pairs?

Question 4 (Text Contrastive Learning) – Optional

Read and test the provided code for this question. The code uses a small dataset of text sentences and defines functions to generate text pairs using a simple data augmentation strategy. It then defines an encoding function to convert these texts into vector representations, as well as a contrastive loss function.

The code further trains a contrastive learning model over several epochs and ultimately produces a trained embedding model capable of encoding any input text sentence into a meaningful vector representation.

Please ensure that you understand the procedure for computing the contrastive loss defined in the function `contrastive_loss(z1, z2, temperature=0.5)`. Then, express this procedure as an algorithm in plain English.